

Literature Review: Software Metrics in Continuous Integration and Continuous Delivery Pipelines

CSE 7319 - Software Metrics and Quality

November 6, 2025

1 Introduction

This literature review examines recent research on internal and external software metrics applied to Continuous Integration and Continuous Delivery (CI/CD) pipelines.

2 Internal Metrics

Internal metrics depend solely on program artifacts and can be measured through static or dynamic analysis. As independent variables in the metrics relationship model, they serve as leading indicators and control mechanisms for achieving external quality goals. Classic categories include size metrics (LOC, function points), complexity metrics (cyclomatic complexity, Halstead measures, data/control flow), and structure metrics (coupling, cohesion). The papers reviewed here focus on how internal metrics are collected and applied within CI/CD pipelines, particularly static analysis quality checks and test coverage measurements.

2.1 Static Code Analysis Metrics

How Open Source Projects Use Static Code Analysis Tools in Continuous Integration Pipelines [1]

2.1.1 Summary

This empirical study investigates how open-source projects integrate static code analysis tools into their continuous integration workflows. The researchers analyzed real-world CI configurations to understand what types of code quality issues cause builds to fail versus generate warnings, and how developers respond to these signals. The study examined projects using tools like Checkstyle, PMD, and FindBugs integrated into CI systems.

A key finding was that the vast majority of build failures triggered by static analysis related to coding standard violations and style issues rather than bug detection. The researchers also found that when static analysis generates warnings or failures, developers typically address the issues directly by fixing the code rather than suppressing warnings or adjusting tool configurations.

2.1.2 Analysis

This paper demonstrates the practical application of static analysis of internal metrics within CI/CD pipelines. The finding that coding style violations constitute the majority of build failures indicates teams using these tools are attempting to enforce code maintainability. This illustrates a predictive relationship, by controlling internal metrics such as code style and structural consistency, teams aim to improve external metrics of maintainability.

The paper validates that adding a quality gate within the build pipeline effectively enforces internal code style requirements. The developers' tendency to fix issues rather than suppress warnings demonstrates that when internal metrics are properly integrated into workflows, they influence behavior and drive quality improvements. From a GQM perspective, these teams have defined maintainability goals, posed questions about code consistency,

and selected metrics such as style violations and complexity checks.

2.2 Test Coverage Metrics

Open Code Coverage Framework: A Consistent and Flexible Framework for Measuring Test Coverage [2]

2.2.1 Summary

This paper addresses a fundamental challenge in measuring test coverage across different programming languages and coverage criteria: the high cost and inconsistency of coverage measurement tools. The researchers observed that while test coverage is a critical quality indicator, each language typically requires separate tools with inconsistent reporting formats and capabilities.

They developed the Open Code Coverage Framework (OCCF), which abstracts common concepts across programming languages using abstract syntax trees (ASTs). By identifying structural commonalities at the AST level, the framework can support multiple languages with a single common solution. The authors demonstrated that their approach reduced implementation requirements by approximately 90% and cut development time for new coverage criteria by 80% or more. The framework supports various coverage types including statement, branch, and path coverage while remaining extensible for custom criteria.

2.2.2 Analysis

This paper addresses a fundamental challenge in measurement theory: ensuring consistent and properly defined metrics across different contexts. Test coverage is an internal metric, and serves as a leading indicator for external quality attributes like reliability and defect density. The framework’s abstraction approach of using ASTs to identify structural commonalities enables uniform measurement of statement, branch, and path coverage across languages. This consistency is essential for establishing valid predictive models: if coverage

is measured differently across components, correlations with external quality metrics become unreliable.

2.3 CI/CD Metrics Visualization

A Roadmap for Using Continuous Integration Environments [4]

2.3.1 Summary

This study investigates how development teams can leverage CI environments to generate and visualize quality metrics. Through interviews with 22 participants across five industrial projects, the researchers identified that 86% of participants found CI-generated data useful for quality attribute evaluation, and 73% valued centralized dashboards for visualizing this data. The study cataloged 14 base metrics and 10 derived metrics spanning seven quality attributes including maintainability, security, performance, and reliability:

- **Version Control Metrics:** No. of Commits
- **Source Code Metrics:** Lines of Code, No. of Source Files
- **Continuous Integration Metrics:** Time Stamps, No. of Builds
- **Static Code Metrics:** Lines of Code, cyclomatic Complexity, Code Smells
- **Artifacts Metrics:** No. of Release Versions, Release Version Tags
- **Test Metrics:** Mean Recovery Time
- **Performance Metrics:** No. of Service Requests, CPU/Memory Usage, No. of Transactions, No. of Vulnerabilities
- **Issue Tracking Metrics:** No. of Tickets

Performance metrics showed the highest adoption at 82% among participants. The paper provides concrete examples of metrics collection pipelines using Prometheus for data gathering and Grafana for visualization. The researchers also propose a three-level maturity model (elementary, intermediate, advanced) for CI environment evolution, finding that 60% of studied projects operated at intermediate maturity levels. Challenges identified include dashboard information overload and insufficient understanding of metrics meanings.

2.3.2 Analysis

This paper directly addresses the metrics usage workflow: after measurement activities collect data, analysis and visualization are essential to make results actionable. The cataloging of 14 base and 10 derived metrics demonstrates the variety of internal and external metrics available in modern CI/CD systems. Performance metrics showing the highest adoption reflects their direct connection to external user experience, while the centralized dashboard preference indicates that integrated visualization of multiple metrics is more valuable than isolated measurements. The identified challenges—information overload and insufficient understanding—highlight a pitfall, where the overcollection of metrics could dilute their importance. This suggests the value for an "Experience Factory" approach where organizations identify which metrics matter and how to interpret them.

3 External Metrics

External metrics reflect quality from the customer and user perspective, measuring the "ilities" (reliability, usability, performance, security, maintainability) that determine system value. As dependent variables in the metrics relationship model, they must be predicted and managed through monitoring of internal metrics.

3.1 DevOps Research and Assessment (DORA) Metrics

A Framework for Automating the Measurement of DORA Metrics [3]

3.1.1 Summary

The DORA metrics (Deployment Frequency, Lead Time for Change, Time to Restore Service, and Change Failure Rate) have become industry-standard indicators of software delivery performance. This paper presents a framework that automatically extracts and calculates DORA metrics from project repositories without requiring specific CI/CD tools or cloud platforms.

The researchers validated their framework by analyzing 304 popular open-source projects on GitHub, generating time-series data that revealed meaningful insights into project performance trends. A key contribution is the project-agnostic design: the framework can operate across diverse development lifecycles and deployment approaches. The analysis showed that significant changes in DORA metrics correlated with major historical events in projects, validating the metrics' ability to detect meaningful shifts in development performance.

3.1.2 Analysis

The DORA metrics represent external quality attributes in the DevOps context, aligned with the Putnam/Myers framework's external metrics of time, quality, and productivity:

- Deployment frequency and lead time reflect productivity and schedule performance
- Change failure rate and time to restore service measure reliability.

The framework's project-agnostic design is significant because it enables consistent measurement across diverse environments. This paper demonstrates that external metrics can be extracted automatically from development artifacts such as commits, deployments, and incidents. The time-series approach is essential for quality improvement: tracking metric

trends over time enables teams to assess whether process changes such as adopting new internal metric thresholds or tools actually improve external outcomes.

3.2 Requirements Traceability

Continuous Integration and Delivery Traceability in Industry: Needs and Practices [5]

3.2.1 Summary

The paper documents the Eiffel framework, developed within Ericsson to provide real-time traceability throughout CI/CD pipelines. The Eiffel approach defines a common protocol to embed traceability into the continuous integration infrastructure, allowing automatic capture of relationships between requirements, commits, builds, tests, and deployments as they occur. This enables immediate visibility into which requirements are affected by changes and what testing has been performed.

3.2.2 Analysis

Requirements traceability connects external requirements like customer needs and specified functionality to internal artifacts such as commits, builds, or tests. The Eiffel framework embeds automatic measurement directly into development workflows, addressing the classic challenge that manual metrics collection is impractical in fast-paced development. This automation enables real-time tracking rather than periodic snapshots, making traceability a useful leading indicator: when a requirement changes, teams immediately know which code and tests are affected.

From a measurement theory perspective, this automated approach produces more reliable data than manual maintenance because it captures actual relationships, like commits linked to requirements, rather than documented intentions. The integration of traceability with CI/CD metrics enables teams to track if specific requirements are fully implemented and

tested.

3.3 Hybrid Development Models

Mathematical Model of the Software Development Process with Hybrid Management Elements [6]

3.3.1 Summary

This very recent paper develops a probabilistic model for hybrid software development life-cycles that combines traditional project management with agile and DevOps practices. The model integrates data from multiple sources including requirements traceability systems, static code analysis tools, and CI/CD pipeline telemetry. Using Graphical Evaluation and Review Technique (GERT) network semantics and fuzzy uncertainty treatment, the model predicts completion timelines and rework requirements.

3.3.2 Analysis

This paper defines a methodology for modelling the relationship between internal and external metrics. By combining data from requirements traceability, static code analysis, and CI/CD telemetry, the model establishes relationships between leading indicators and outcome variables. The probabilistic approach using GERT enables the prediction of external quality attributes like schedule and rework, from internal metrics.

4 Conclusion

The reviewed papers demonstrate how fundamental software metrics principles apply in modern CI/CD environments. Several key themes connect the research to metrics theory:

Linking Internal and External Metrics: The papers illustrate the predictive relationship between internal metrics and external quality outcomes. CI/CD pipelines provide

infrastructure for measuring both metric types automatically and establishing their correlations, as demonstrated in the DORA framework and Semenov’s probabilistic model.

Automation Enables Measurement Theory: The automated frameworks reviewed (DORA, Eiffel, OCCF) ensure uniform measurement across diverse contexts, enabling reliable predictive models. This addresses the classical challenge that inconsistent measurement produces unreliable correlations between internal and external metrics.

From Measurement to Control: The papers demonstrate the following metrics usage workflow: **Automated collection** → **Dashboards/predictive models** → **Action/process improvements** → **Validation**. This progression shows metrics evolving from descriptive statistics into active controls that guide development.

Holistic Quality Assessment: Integrating multiple metric types—internal code quality, test adequacy, requirements traceability, and external delivery performance—provides richer insights than isolated measurements. The Semenov model exemplifies this by synthesizing requirements, static analysis, and CI/CD data into unified predictions.

These papers confirm that CI/CD environments enable the vision of comprehensive, automated metrics collection, aggregation, and display. They demonstrate that when properly implemented, metrics frameworks support continuous quality improvement by making both internal leading indicators and external outcome measures visible, actionable, and connected through validated predictive relationships.

References

- [1] F. Zampetti, S. Scalabrino, R. Oliveto, G. Canfora, and M. Di Penta, “How open source projects use static code analysis tools in continuous integration pipelines,” in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. Buenos Aires, Argentina: IEEE, 2017, pp. 334–344.
- [2] K. Sakamoto, H. Washizaki, and Y. Fukazawa, “Open code coverage framework: A consis-

tent and flexible framework for measuring test coverage supporting multiple programming languages,” in *2010 10th International Conference on Quality Software*. Zhangjiajie, China: IEEE, 2010, pp. 262–269.

- [3] B. Wilkes, A. M. P. Milani, and M.-A. Storey, “A framework for automating the measurement of devops research and assessment (dora) metrics,” in *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. Bogotá, Colombia: IEEE, 2023, pp. 62–72.
- [4] L. Yu, E. Alégroth, P. Chatzipetrou, and T. Gorschek, “A roadmap for using continuous integration environments,” *Communications of the ACM*, vol. 67, no. 6, pp. 82–90, June 2024.
- [5] D. Ståhl, K. Hallén, and J. Bosch, “Continuous integration and delivery traceability in industry: Needs and practices,” in *2016 42nd Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. Limassol, Cyprus: IEEE, 2016, pp. 68–72.
- [6] S. Semenov, V. Tsukur, V. Molokanova, M. Muchacki, G. Litawa, M. Mozhaiev, and I. Petrovska, “Mathematical model of the software development process with hybrid management elements,” *Applied Sciences*, vol. 15, no. 21, p. 11667, 2025.